



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent stores a lookup table that contains an action for every possible percept sequence. While this works for very small environments, it is impossible for complex environments such as Chess because the number of possible percept sequences grows exponentially, a problem known as combinatorial explosion.

As the agent's lifetime increases, the percept sequence becomes longer and the number of possible histories increases dramatically. The lookup table would require an enormous amount of memory and would take an impractical amount of time to search. Therefore, creating and storing a complete table for every possible situation is mathematically impossible for real-world problems.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The Condition-Action Rules in the `SimpleReflexAgent` are represented by the IF-THEN statements inside the `sense_and_act()` method. The statement `if percept["food_here"]:` `return "Suck"` represents the rule IF food is present in the current location, THEN collect the food. Similarly, the statement `elif percept["wall_ahead"]:` `return "TurnLeft"` represents the rule IF there is a wall ahead, THEN turn left. The final statement, `else:` `return "Forward"`, represents the rule IF there is no food and no wall ahead, THEN move forward. These IF-THEN statements directly implement the Condition-Action architecture discussed in the lecture because the agent makes decisions only by matching the current percept with predefined rules. It does not store previous experiences, reason about future consequences, or maintain any internal state. The agent simply observes the current environment condition and performs the corresponding action.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The SimpleReflexAgent gets stuck because it operates only using the current percept and has no memory of previous situations. Due to partial observability, the agent cannot access complete information about the environment, such as its previous locations or the paths it has already explored.

For example, when the agent reaches a wall, it follows the rule "IF wall_ahead THEN turn left." However, after turning, it may eventually return to the same location and receive the same percept again. Since the agent has no percept history, it cannot recognize that it has already visited that location or that it is repeating the same sequence of actions. As a result, it continues applying the same rules and can become trapped in an infinite cycle. The failure occurs because the Simple Reflex Agent assumes that the current percept contains enough information to make the correct decision, but in partially observable environments, important information about the hidden state of the world is unavailable.



4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The ModelBasedAgent improves the SimpleReflexAgent by maintaining an internal state through memory. In the `__init__()` method, variables such as `self.visited_cells`, `self.last_action`, and `self.position` are initialized to store information about previously visited locations, the previous action performed, and the agent's estimated position. The Transition Model is handled by updating this internal state based on the previous action taken by the agent. For example, when the previous action is "Forward", the agent updates its estimated position according to its current direction, and when the previous action is "TurnLeft", it updates its direction. This allows the agent to predict how its actions change the state of the environment. The Sensor Model is represented by the percept information received from the environment, such as `wall_ahead` and `food_here`, which provide information about the current situation. Before selecting a new action, the agent first updates its memory, records the current state, and checks whether the next possible movement has already been visited. Therefore, unlike the SimpleReflexAgent, the ModelBasedAgent can use past experiences to recognize repeated paths, avoid loops, and make better decisions in a partially observable environment.



Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING